Enhanced WhatsApp Shop Bot System Specification

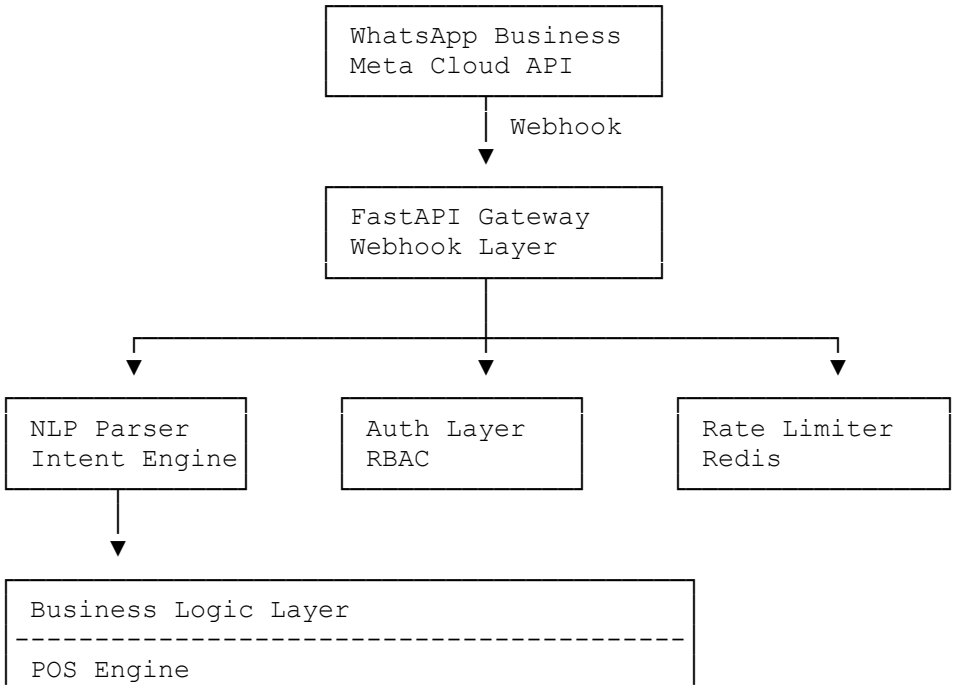
Project Vision

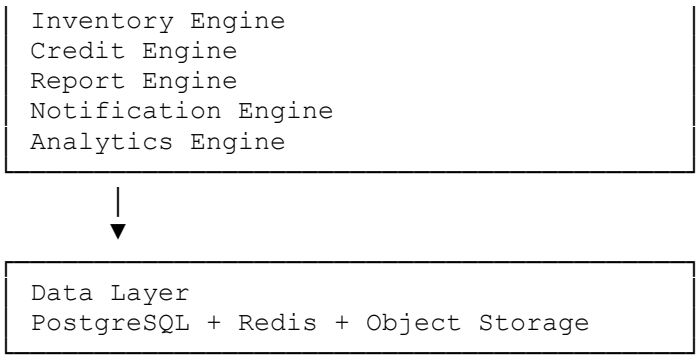
A production-grade WhatsApp commerce assistant for small shops, supermarkets, kiosks, pharmacies, wholesalers, and agents.

The bot should support:

- Natural-language business operations
- Multi-user shop management
- Inventory + POS
- Credit/debt tracking
- Reports & analytics
- Offline-tolerant workflows
- AI-assisted command understanding
- Scalable multi-tenant architecture
- Audit logging & security

Recommended High-Level Architecture





Enhanced Project Structure

```
shop-bot/
│
├── app/
│   ├── main.py
│   │
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── logger.py
│   │   ├── exceptions.py
│   │   ├── middleware.py
│   │   └── constants.py
│   │
│   ├── api/
│   │   ├── webhook/
│   │   │   ├── router.py
│   │   │   ├── verify.py
│   │   │   └── handlers.py
│   │   │
│   │   ├── admin/
│   │   │   ├── router.py
│   │   │   └── dashboard.py
│   │   │
│   │   └── health/
│   │       └── router.py
│   │
│   ├── parser/
│   │   ├── command_parser.py
│   │   ├── intents.py
│   │   ├── validators.py
│   │   ├── ai_fallback.py
│   │   └── synonyms.py
│   │
│   ├── engines/
│   │   ├── pos.py
│   │   ├── inventory.py
│   │   ├── debt.py
│   │   ├── reports.py
│   │   ├── analytics.py
```

```
│   │   └── notifications.py
│   │
│   ├── services/
│   │   ├── whatsapp.py
│   │   ├── session.py
│   │   ├── redis.py
│   │   ├── scheduler.py
│   │   ├── receipts.py
│   │   ├── exports.py
│   │   ├── audit.py
│   │   └── cache.py
│   │
│   ├── database/
│   │   ├── connection.py
│   │   ├── base.py
│   │   ├── models/
│   │   │   ├── product.py
│   │   │   ├── sale.py
│   │   │   ├── debt.py
│   │   │   ├── payment.py
│   │   │   ├── user.py
│   │   │   ├── audit_log.py
│   │   │   └── shop.py
│   │   │
│   │   └── migrations/
│   │
│   ├── schemas/
│   │   ├── product.py
│   │   ├── sales.py
│   │   ├── debt.py
│   │   └── reports.py
│   │
│   ├── workers/
│   │   ├── low_stock_worker.py
│   │   ├── report_worker.py
│   │   └── cleanup_worker.py
│   │
│   ├── tests/
│   │   ├── test_parser.py
│   │   ├── test_sales.py
│   │   ├── test_inventory.py
│   │   └── test_reports.py
│   │
│   └── utils/
│       ├── currency.py
│       ├── dates.py
│       ├── formatter.py
│       └── helpers.py
│
├── docker/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── nginx.conf
│
├── scripts/
│   ├── seed_data.py
│   └── backup.sh
```

```
├── .env
├── requirements.txt
├── alembic.ini
└── README.md
```

Enhanced Feature Set

1. Smart NLP Command Understanding

Instead of strict regex only:

```
sell 2 soda
sold 3 cocacola to john
ongeza stock ya maji 20
nimelipa deni la john 5000
```

The parser should support:

- Swahili + English
- Misspellings
- Synonyms
- Shortcuts
- Context continuation

Example:

```
User: sell 2 soda 1000
Bot: Customer name?
User: john
```

Redis session keeps temporary conversation state.

Intent Categories

Category	Examples
Sales	sell, sold, uza
Inventory	stock, bidhaa
Debt	debt, deni
Reports	report, mauzo
Payments	paid, lipa

Category	Examples
Product management	new, add
Analytics	top selling
Alerts	low stock

Advanced Command Examples

Sales

```
sell 2 soda 1500
sell 1 rice 2500 john
uza maji 3
```

Inventory

```
stock add sugar 50
restock soda 100
stock all
```

Product Management

```
new sugar 3500 100
price sugar 4000
delete sugar
```

Reports

```
report today
report week
top products
sales john
profit today
```

Debt

```
debt john
paid john 5000
credit report
```

Multi-Tenant Shop Support

Support multiple businesses in one deployment.

shops table

```
CREATE TABLE shops (
  id              UUID PRIMARY KEY,
  name            VARCHAR(120) NOT NULL,
  currency        VARCHAR(10) DEFAULT 'TZS',
  timezone        VARCHAR(50) DEFAULT 'Africa/Dar_es_Salaam',
  created_at      TIMESTAMP DEFAULT NOW()
);
```

Every major table references:

```
shop_id UUID REFERENCES shops(id)
```

This enables:

- SaaS deployment
- Many shops on one server
- Subscription billing later

Improved Database Design

Products

```
CREATE TABLE products (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id         UUID REFERENCES shops(id),
  name            VARCHAR(120) NOT NULL,
  sku             VARCHAR(50),
  barcode         VARCHAR(100),
  category        VARCHAR(100),
  price           NUMERIC(12,2) NOT NULL,
  cost_price      NUMERIC(12,2),
  stock_qty       INTEGER DEFAULT 0,
  reorder_at      INTEGER DEFAULT 5,
  unit            VARCHAR(20) DEFAULT 'pcs',
  is_active       BOOLEAN DEFAULT TRUE,
  created_at      TIMESTAMP DEFAULT NOW(),
  updated_at      TIMESTAMP DEFAULT NOW()
);
```

Sales

```
CREATE TABLE sales (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id         UUID REFERENCES shops(id),
```

```
  receipt_no      VARCHAR(50) UNIQUE,
  customer_name   VARCHAR(120),
  total_amount    NUMERIC(12,2),
  payment_method  VARCHAR(30),
  is_credit       BOOLEAN DEFAULT FALSE,
  sold_by         UUID REFERENCES users(id),
  sold_at         TIMESTAMP DEFAULT NOW()
);
```

Sale Items

```
CREATE TABLE sale_items (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sale_id         UUID REFERENCES sales(id),
  product_id      UUID REFERENCES products(id),
  qty             INTEGER NOT NULL,
  unit_price      NUMERIC(12,2),
  total           NUMERIC(12,2)
);
```

Credit Ledger

```
CREATE TABLE credit_ledger (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id         UUID REFERENCES shops(id),
  customer_name   VARCHAR(120),
  amount          NUMERIC(12,2),
  type            VARCHAR(20), -- debt/payment
  note            TEXT,
  created_by      UUID REFERENCES users(id),
  created_at      TIMESTAMP DEFAULT NOW()
);
```

Audit Logs

```
CREATE TABLE audit_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID REFERENCES users(id),
  action          VARCHAR(100),
  entity_type     VARCHAR(50),
  entity_id       VARCHAR(100),
  metadata        JSONB,
  created_at      TIMESTAMP DEFAULT NOW()
);
```

Security Enhancements

Role-Based Access Control

Roles:

Role	Access
owner	Full access
manager	Reports + inventory
cashier	Sales only
viewer	Read-only

Phone Verification

Only approved WhatsApp numbers may interact.

```
ALLOWED_PHONES=+2557..., +2556...
```

Rate Limiting

Prevent spam/abuse.

Redis-based:

```
5 requests / second
```

Audit Trail

Track:

- Deleted products
- Price changes
- Debt edits
- Manual stock changes

AI Fallback Parsing

If regex fails:

```
response = await ai_parse(text)
```

Example:

```
"john amelipa 3000"
→ payment intent
```

Can use:

- OpenAI
- Local LLM
- Ollama

Notification System

Automated Alerts

Low Stock

```
⚠ Low stock:
- Sugar: 2 left
- Soda: 1 left
```

Daily Report

```
 Daily Summary
Sales: TZS 450,000
Profit: TZS 87,000
Credits: TZS 20,000
```

Debt Reminder

```
Reminder:
John owes TZS 25,000
```

Analytics Engine

KPIs

- Total sales
- Profit margin
- Fast-moving products
- Slow-moving products

- Best customers
- Debt trends

Receipt Generation

Generate WhatsApp-friendly receipts.

```
 RECEIPT #1024

2 x Soda      3,000
1 x Bread     1,500

TOTAL:        4,500
Paid Cash

Thank you 🙏
```

Optional:

- PDF receipts
- Thermal printer integration
- QR code receipts

Redis Usage

Redis Responsibilities

Feature	Purpose
Sessions	Multi-step flows
Rate limiting	Abuse protection
Caching	Faster reports
Job queue	Background workers
Temporary locks	Prevent duplicate sales

Background Workers

Use:

- Celery
- RQ
- Dramatiq

Jobs:

Worker	Purpose
report_worker	Scheduled reports
stock_worker	Low-stock checks
cleanup_worker	Remove stale sessions

Suggested API Endpoints

Webhook

```
POST /webhook
GET /webhook
```

Admin APIs

```
GET /admin/reports
GET /admin/products
POST /admin/products
GET /admin/sales
```

Better Parsing Strategy

Instead of regex-only:

Layered Parser

```
1. Exact regex
2. Synonym mapping
3. NLP intent detection
4. AI fallback
5. Human-friendly clarification
```

Example Smart Flow

```
User: sell soda

Bot:
How many soda sold?

User:
3

Bot:
Price per item?

User:
1500

Bot:
Recorded:
3 soda sold for TZS 4,500
```

Production Deployment Architecture

Recommended Stack

Component	Tech
API	FastAPI
Database	PostgreSQL
Cache	Redis
Queue	Celery/RQ
Reverse proxy	Nginx
SSL	Let's Encrypt
Containerization	Docker
Monitoring	Prometheus + Grafana
Logs	Loki / ELK

Docker Compose

```
services:
  api:
    build: .
    ports:
      - "8000:8000"

  postgres:
    image: postgres:16
```

```
  redis:
    image: redis:7

  nginx:
    image: nginx
```

Monitoring & Observability

Add:

- Structured logging
- Error tracking
- Performance monitoring
- Request tracing

Recommended:

- Sentry
- Prometheus
- Grafana

Scalability Roadmap

Phase 1

- WhatsApp commands
- POS
- Inventory

Phase 2

- AI NLP
- Dashboard
- Analytics

Phase 3

- Multi-tenant SaaS
- Subscription billing
- Mobile app

Phase 4

- Voice notes parsing
- OCR receipts
- AI forecasting
- Supplier automation

Recommended Future Features

Mobile Dashboard

- React Native / Flutter

Web Dashboard

- React.js Admin Panel

Voice Commands

```
"uza soda mbili kwa john"
```

Speech-to-text parsing.

Suggested Additional Modules

Module	Purpose
loyalty.py	Customer rewards
suppliers.py	Supplier tracking
expenses.py	Business expenses
forecasting.py	Stock prediction
backup.py	Automated backups
invoices.py	Invoice generation

Recommended Improvements to Your Current Version

Replace

Old

```
(r"^sell (\d+) (\w+) (\d+)(?:\s+(\w+))?$", "sell")
```

Better

Named groups + flexible parser:

```
r"^sell\s+(?P<qty>\d+)\s+(?P<product>.+?)\s+(?P<price>\d+)(?:\s+(?P<customer>.+))?$"
```

Add Validation Layer

Validate:

- Product exists
- Stock available
- Price valid
- User authorized

Add Transaction Safety

Use DB transactions:

```
async with session.begin():
```

Prevents:

- Duplicate sales
- Stock inconsistencies
- Partial writes

Final Recommended Stack

Layer	Recommendation
Backend	FastAPI
DB	PostgreSQL
Cache	Redis
Queue	Celery
ORM	SQLAlchemy Async
Auth	JWT + phone allowlist
Infra	Docker + Nginx
Monitoring	Grafana
AI	OpenAI/Ollama
Frontend	React.js
Mobile	Flutter